

Large language model-based optical network log analysis using LLaMA2 with instruction tuning

YUE PANG,¹ MIN ZHANG,¹  YANLI LIU,¹ XIANGBIN LI,¹ YIDI WANG,¹  YAHANG HUAN,¹ ZHUO LIU,² JIN LI,¹  AND DANSHI WANG^{1,*} 

¹State Key Laboratory of Information Photonics and Optical Communications, Beijing University of Posts and Telecommunications, Beijing 100876, China

²China Mobile Communications Corporation, Beijing 100033, China

*danshi_wang@bupt.edu.cn

Received 17 April 2024; revised 12 July 2024; accepted 30 September 2024; published 22 October 2024

The optical network encompasses numerous devices and links, generating a significant volume of logs. Analyzing these logs is significant for network optimization, failure diagnosis, and health monitoring. However, the large-scale and diverse formats of optical network logs present several challenges, including the high cost and difficulty of manual processing, insufficient semantic understanding in existing analysis methods, and the strict requirements for data security and privacy. Generative artificial intelligence (GAI) with powerful language understanding and generation capabilities has the potential to address these challenges. Large language models (LLMs) as a concrete realization of GAI are well-suited for analyzing DCI logs, replacing human experts and enhancing accuracy. Additionally, LLMs enable intelligent interactions with network administrators, automating tasks and improving operational efficiency. Moreover, fine-tuning with open-source LLMs protects data privacy and enhances log analysis accuracy. Therefore, we introduce LLMs and propose a log analysis method with instruction tuning using LLaMA2 for log parsing, anomaly detection and classification, anomaly analysis, and report generation. Real log data extracted from the field-deployed network was used to design and construct instruction tuning datasets. We utilized the dataset for instruction tuning and demonstrated and evaluated the effectiveness of the proposed scheme. The results indicate that this scheme improves the performance of log analysis tasks, especially a 14% improvement in exact match rate for log parsing, a 13% improvement in F1-score for anomaly detection and classification, and a 23% improvement in usability for anomaly analysis, compared with the best baselines. © 2024 Optica Publishing Group

<https://doi.org/10.1364/JOCN.527874>

1. INTRODUCTION

Optical networks have advantages such as high bandwidth, low latency, and long transmission distances. They support a wide range of critical services and infrastructure. The secure operation of optical networks is crucial for modern society [1,2]. The optical network involves a large number of optical network devices and links, and a vast amount of log data is generated daily. These optical network logs are invaluable resources. Optical network logs contain information about the network's operation, recording events, states, and anomalous activities, which can be utilized for tracking network activities, understanding network status, and managing system security [3,4]. Moreover, due to the high frequency of log generation, analyzing logs enables more efficient and accurate network behavior understanding, failure diagnosis, health monitoring, anomaly detection, and root cause analysis [5–12]. This contributes to further network optimization, diagnostics, and maintenance [13–15]. In summary, in-depth exploration and analysis of optical network logs are crucial for ensuring smooth

network operation, efficient resource utilization, and meeting ever-growing demands.

However, the exploration of optical network logs currently remains insufficient, resulting in their value not being fully utilized. The analysis of optical network logs faces several challenges. First, optical network logs have large volumes and diverse formats. Manual monitoring and processing of optical network logs require advanced expertise, are time-consuming and costly, and are prone to misunderstandings and biases. This presents challenges for efficient and reliable log analysis. Additionally, optical network logs are presented in textual form. But, existing natural language processing (NLP) methods have limited semantic understanding capabilities, which require stronger language models to acquire semantic representations of the logs. Furthermore, optical network operators possess rich proprietary log data that cannot be transmitted to third-party platforms for analysis, thus requiring strict data security and privacy. These challenges limit the full utilization of optical network logs. Therefore, new methods and

technologies need to be developed to address these challenges and unlock the full potential of optical network logs.

The field of generative artificial intelligence (GAI) has witnessed groundbreaking advancements, with models like OpenAI's GPT series (e.g., ChatGPT [16] and GPT-4 [17]) demonstrating the ability to engage in fluid conversations with humans and generate coherent text. These large language models (LLMs) can process large amounts of text data, learn patterns and regularities, and generate human-like text [18–20]. These characteristics make a LLM a suitable tool for analyzing optical network logs and extracting valuable insights. First, LLMs can replace manual processing of large and complex logs. This reduces the need for advanced expertise and lowers the risks of errors and biases. Second, LLMs offer enhanced language comprehension capabilities, enabling them to capture the semantic meaning of log entries and achieve more accurate analysis. Furthermore, LLMs can intelligently interact with network administrators in natural language and automate the execution of complex tasks by integrating with external tools. This capability allows network administrators to simply express their needs, questions, or instructions without the need to be familiar with specific commands or programming languages, thereby improving operational efficiency and flexibility. Finally, OpenAI has not disclosed detailed information about their training strategy or weight parameters, which necessitates sending data to OpenAI's platform for usage and prohibits fine-tuning for downstream tasks. Hence, LLaMA2 [21] emerged as a series of open-source pretrained and fine-tuned LLMs. LLaMA2 allows researchers to train their customized models on custom datasets. Open-source LLMs (such as LLaMA2) allow for fine-tuning using local log data, ensuring the privacy and security of sensitive log data while greatly enhancing the accuracy of log analysis.

In recent years, numerous studies have emerged focusing on the analysis of computer system logs. These studies employ rule-based approaches [22,23], machine learning-based methods [24], and deep learning-based techniques [25–27] for log parsing and anomaly detection. Currently, the exploration of optical network logs is still in its infancy. Therefore, we focus on the valuable resource of optical network logs and introduce a LLM to mine it. We propose a LLM-based optical network log analysis method (ONLA-LLM) with instruction tuning using LLaMA2, which achieves log parsing, anomaly detection and classification, anomaly analysis, and report generation. We have designed instruction tuning to make LLMs suitable for optical network log analysis, particularly by integrating optical network log data collected from a field-deployed network into instruction tuning datasets. Instruction tuning demonstrations were conducted on LLaMA2 to achieve log analysis tasks. Meanwhile, the integration with LangChain achieves the automation of the report generation process. Finally, this paper provides a comprehensive evaluation of the demonstration results. The results indicate that instruction tuning improves the performance of log analysis tasks, especially a 14% improvement in exact match rate for log parsing, a 13% improvement in F1-score for anomaly detection and classification, and a 23% improvement in usability for anomaly analysis, compared with the best baselines. Valuable insights can be deeply explored from optical network logs by fully

leveraging the potential of the LLM, which improve network performance, reliability, and security.

2. ONLA-LLM

A. Framework

Optical network logs typically contain information about network performance, device status, configuration changes, operational records, and security events. Abnormal logs among optical network logs include system configuration errors, abnormal operational behaviors, potential performance issues, network connectivity problems, and service outages, which should draw the attention of network administrators. These logs are represented in text format, usually including time-stamps, device identifiers, event identifiers, and specific content. In-depth exploration of these logs contributes to improving the reliability, availability, and security of optical networks.

However, traditional analysis methods are error-prone and fail to fully explore the potential information within the logs. Therefore, we introduce the LLM to enhance the capability and enrich the purpose of log analysis in optical networks. We employ the open-sourced LLaMA2 model and fine-tune it to understand the domain terminologies and knowledge in optical networks. This facilitates log parsing, anomaly detection and classification, anomaly analysis, and report generation, as illustrated in Fig. 1.

In ONLA-LLM, we first label historical log data based on domain experts' expertise to inject domain knowledge. Domain experts have a deep understanding of the log format, structure, and parsing rules of the optical network. They can identify abnormal patterns and behaviors in optical network logs and analyze the exceptional situations based on their in-depth knowledge of the optical network. Specifically, domain experts respectively label the data for log parsing, anomaly detection and classification, and anomaly analysis tasks. Next, we construct corresponding instructions for each task, which include information such as task description, specific requirements, and format statements. The purpose of these instructions is to guide the model in analyzing and processing the input data. Instruction tuning datasets are built by merging the constructed instructions and the labeled data. By fine-tuning the LLM on these datasets, the model can learn the domain knowledge and task requirements related to optical network logs to improve its accuracy of log analysis.

During the log analysis process, ONLA-LLM will be used to parse the logs and convert them into a structured data format. Next, log anomaly detection and classification are performed. The ONLA-LLM identifies abnormal logs and classifies them into specific anomaly types. Finally, anomaly analysis is performed on the abnormal logs to investigate the causes and solutions. According to the needs of network administrators, ONLA-LLM can be used to automatically generate reports on log analysis results. Optical network logs can be efficiently analyzed through the above process. Among them, the LLM provides the ability to quickly locate and resolve abnormal situations, which can help network administrators optimize

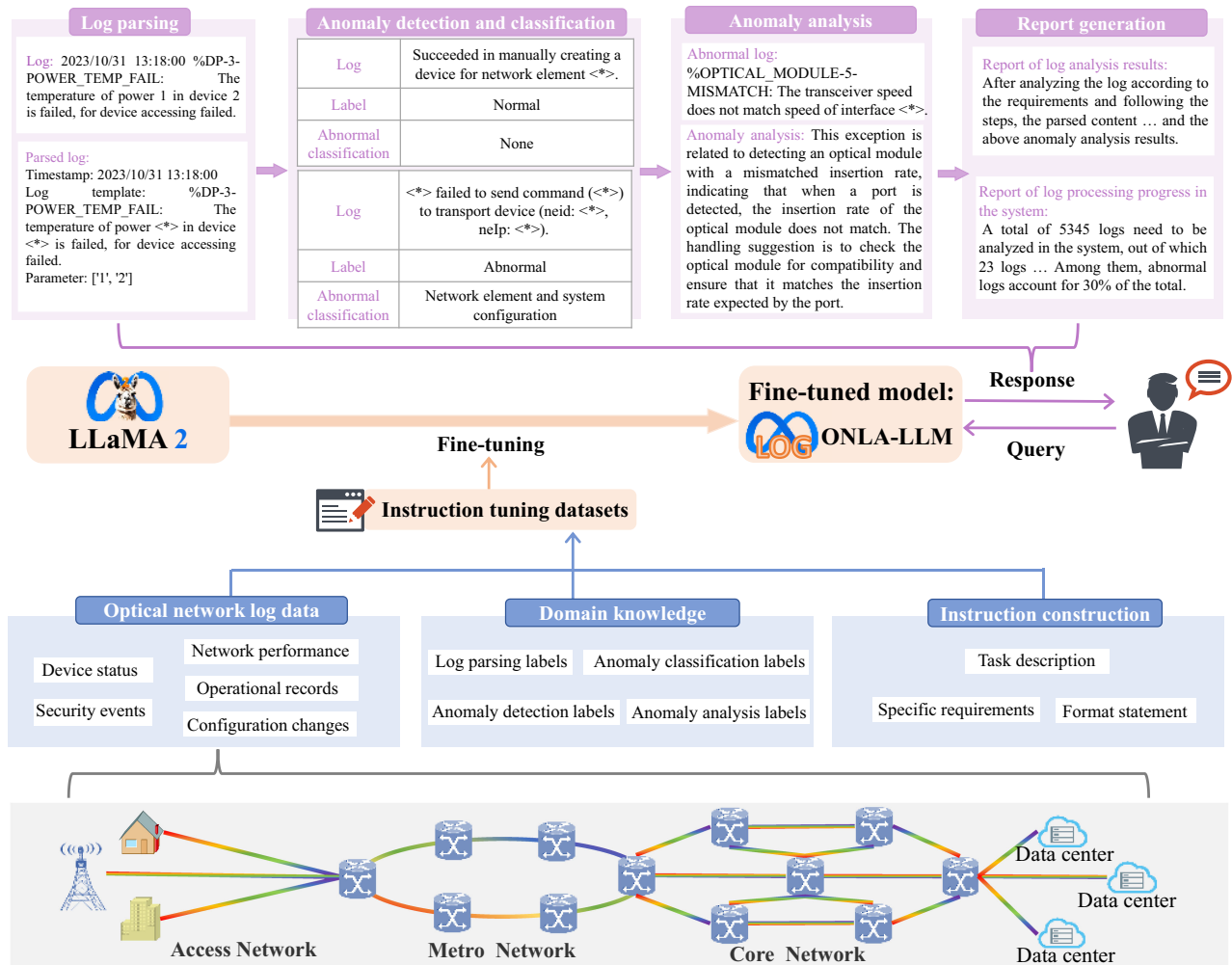


Fig. 1. Constructing optical network log datasets and instruction tuning LLaMA2 to achieve log parsing, anomaly detection and classification, anomaly analysis, and report generation.

network performance, enhance network security, and improve operational efficiency.

B. Log Parsing

Log parsing is an essential initial step in many log-based system diagnostic tasks. The goal of log parsing is to extract common segments (templates) and unique segments (variables) from raw logs, transforming semi-structured log messages into structured log messages [28–30]. Structured logs are easier to manage, allowing for effortless storage, indexing, and aggregation in databases. Large models are suitable for the log parsing process since it involves understanding and processing text. For the log parsing of optical network logs, three main steps are involved: extracting log timestamps, extracting constants as log templates from log messages, and extracting variable parts (parameters), as shown in Fig. 1.

C. Anomaly Detection and Classification

Anomalies in log data usually indicate system failures, security vulnerabilities, or operational failures. The log anomaly

detection task is to identify log messages that may indicate system problems during operation [31]. ONLA-LLM can identify abnormal text patterns, error descriptions, error codes, and other information to determine the presence of anomalies and return a label of normal or abnormal. Once abnormal logs are identified, they can be further classified into different categories, such as network element and system configuration, monitoring information, network performance tips, and so on. This makes it easier for network administrators to identify different types of problems, facilitating quick problem localization and resolution.

D. Anomaly Analysis

Interpretability of abnormal logs is a significant requirement in operations and management. Anomaly analysis can provide valuable insights into the root causes and recommended actions for abnormal logs. ONLA-LLM can learn the domain knowledge and context of anomaly analysis by pretraining or fine-tuning LLMs using optical network anomaly analysis-related data. This empowers ONLA-LLM to accurately understand key information in the descriptions of abnormal

events and provide insights. This reduces the requirement for network administrators' professional knowledge reserve and helps them quickly understand the problems. As a result, it saves a lot of costs in terms of training, manpower, and time while improving the accuracy and efficiency of anomaly resolution.

E. Report Generation

LLMs have the ability to generate coherent and reasonable text based on given prompts. Therefore, the network administrator describes the information they want to obtain as natural language input to the ONLA-LLM. The model can autonomously determine the objectives, plan the steps, invoke appropriate tools, and summarize the findings in a report. The report includes an overview of abnormal situations, data visualization, and a summary of log analysis. This process can automate the log analysis process in the form of natural language interaction, reducing the effort of manual retrieval of information and improving efficiency. Ultimately, it provides clear and understandable references to network administrators in natural language form. This process can be implemented using LangChain, and the specific implementation details can be found in Section 3.

3. METHODOLOGY

A. Instruction Tuning Dataset Construction

Fine-tuning is the process of further training a pretrained LLM. It involves adjusting the model's parameters on specific task or domain data to improve its performance in that particular task

or domain. Instruction tuning can be seen as a special form of supervised fine-tuning, where the LLM is trained to generate outputs based on instructional inputs. Fine-tuned models based on LLaMA2 have been utilized in various domains such as medicine, law, and finance, demonstrating their potential in a wide range of practical applications [32].

Instruction tuning is a process of further training LLMs on a dataset consisting of (instruction, output) pairs. It helps LLMs better adapt to downstream tasks and enhances generalization ability [33]. The quality of the instruction dataset is crucial for successful instruction tuning. The dataset typically includes instruction, input (optional), and output fields. Instructions represent human instructions to guide the model's behavior, and the wording and style of the instructions are essential for correctly guiding the model's behavior. Inputs provide additional contextual information, while outputs represent the desired output following the instructions [34].

In order to achieve better results in each task of log analysis, we have specifically designed corresponding instruction templates for log parsing, anomaly detection and classification, and anomaly analysis, as shown in Fig. 2. The instruction field usually includes task description, specific requirements, and format statements. These fields can be flexibly combined according to the needs. The task description field provides detailed information about the task or desired behavior to help the model understand the objectives and requirements of the task. The specific requirements field provides detailed requirements and constraints based on the task scenario, ensuring that the model generates expected results. The format statement field describes the expected output format to ensure the standardization and usability of the output. The instructions should

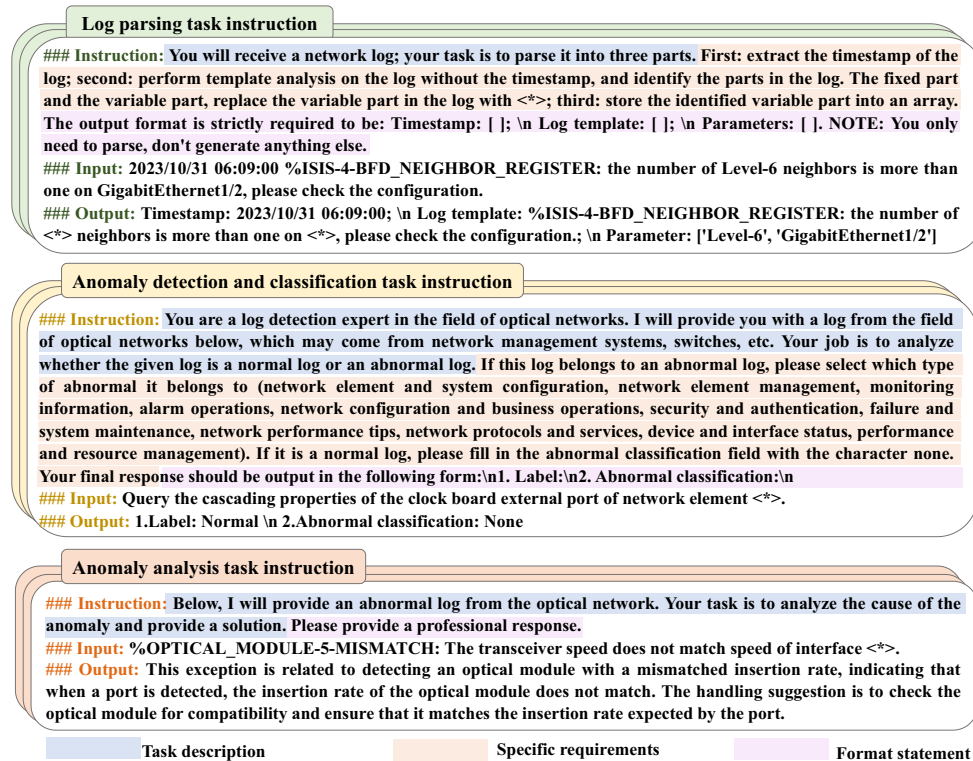


Fig. 2. Templates of instruction tuning datasets for log parsing, anomaly detection and classification, and anomaly analysis.

be fluent, clear, and concise to facilitate the understanding of the LLM.

Taking log parsing as an example, as shown in Fig. 2, the instruction first states that the task is to parse the log into three parts. Then, it provides specific requirements for the log parsing process, such as how to identify and fill in variable and fixed parts. Finally, the output format requirements are clearly defined. The input field is retained to provide log information. The output field is populated with log parsing labels, anomaly detection and classification labels, and anomaly analysis labels based on expert annotations.

B. QLoRA-Based Instruction Tuning

As the size of LLMs continues to grow exponentially, it is becoming increasingly impractical to fine-tune every parameter. In contrast, parameter-efficient fine-tuning (PEFT) methods freeze most of the parameters in the base model and only optimize a small subset, which significantly reduces the computational and memory cost. To facilitate training and inference on limited computational resources while achieving effective fine-tuning, this paper applies the QLoRA [35] technique in PEFT to the fine-tuning of ONLA-LLM to realize various log analysis tasks, as shown in Fig. 3. Overall, QLoRA is actually a quantize and LoRA technique. QLoRA uses a low-precision storage data type (NF4) to compress the pretrained LLM. LoRA achieves efficient adaptation of LLMs through low-rank updates.

In LoRA, a parameter update for a pretrained weight matrix is specified by a product of two low-rank matrices W_A and W_B :

$$\delta W = W_A W_B, \quad (1)$$

where, assuming the pretrained-model parameters, W_0 is a $d \times k$ matrix, and W_A and W_B are matrices of trainable parameters with dimensions $d \times r$ and $r \times k$, respectively, where the

rank $r \ll \min(d, k)$. The pretrained-model parameters W_0 are frozen during training and do not receive gradient updates. δ represents the change or update in the weight matrix. Both W_0 and δW are multiplied with the same input; hence, the forward pass for $h = W_0 x$ is altered as [35]

$$h = W_0 x + \delta W x = W_0 x + W_A W_B x. \quad (2)$$

x is a d -dimensional input vector, and h is a k -dimensional output vector. The matrix W_A is initially set to zero while the matrix W_B is started with a random Gaussian initialization. Therefore, δW is zero when the fine-tuning process begins. During training, $\delta W x$ is scaled by α/r , where α is a constant in r [35]. After training, the trainable parameters can be combined with the original weight matrix W_0 by adding the matrix $W_A W_B$ to W_0 [36]. Compared with complete fine-tuning, LoRA significantly reduces the number of trainable parameters while ensuring training effectiveness.

C. Report Generation with LangChain

LangChain is a framework for developing applications for LLMs. It provides universal interfaces, diverse tools, and modular components for various types of LLMs. This framework facilitates the rapid development of complex LLM applications for developers [37]. LangChain provides a set of tools, components, and interfaces. We mainly use agents and tools in LangChain. Different tools can be defined and written to complete specific tasks. Agents are the entities that drive decision-making in LangChain. They have access to a set of tools and can decide which tool to call based on the user input [38]. It is necessary to integrate knowledge and tool results to provide to the LLM to generate a report. LangChain can play a role in this process.

We have defined “Log_parsing,” “Anomaly_detection_classification,” and “Anomaly_analysis” tools. These tools can run respective fine-tuned models to perform log parsing, anomaly detection and classification, and anomaly analysis, generating output results. Additionally, we have defined “Retrieve” and “Drawing” tools. The “Retrieve” tool retrieves relevant data about the current log processing progress from the database. The “Drawing” tool generates a visualization image based on the input data.

LangChain encapsulates and implements the ReAct framework through the Agent class. When the agent receives a task, the ReAct framework prompts the LLMs to generate inference traces and actions for the task and then the agent calls relevant tools based on the actions. In this paper, we primarily demonstrate the report generation capability of the LLM by generating reports for analyzing a single log and summarizing log processing progress results, as shown in Fig. 4. Network administrators only need to input their requirements in the form of natural language to automatically complete a series of actions to generate the report. The LLM can analyze the input content and understand the process step through the ReAct framework, and then decide which tools to invoke according to the definition of the tool. Finally, the LLM summarizes the obtained results and generates a readable report presented in natural language.

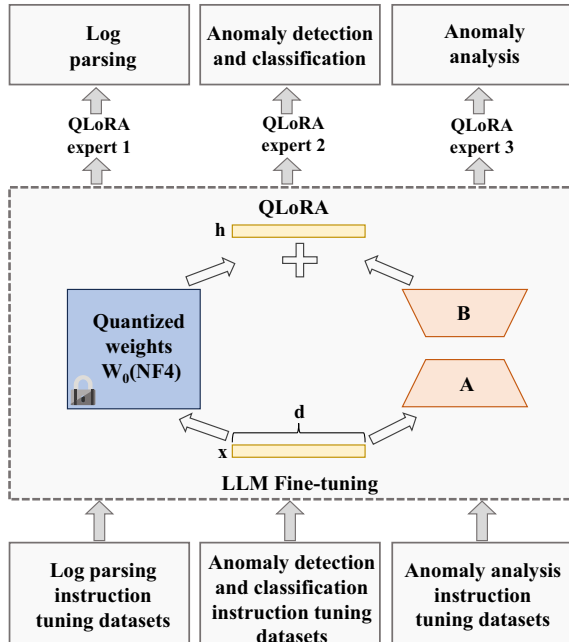


Fig. 3. Applying the QLoRA technique to fine-tune LLaMA2 for various log analysis tasks.

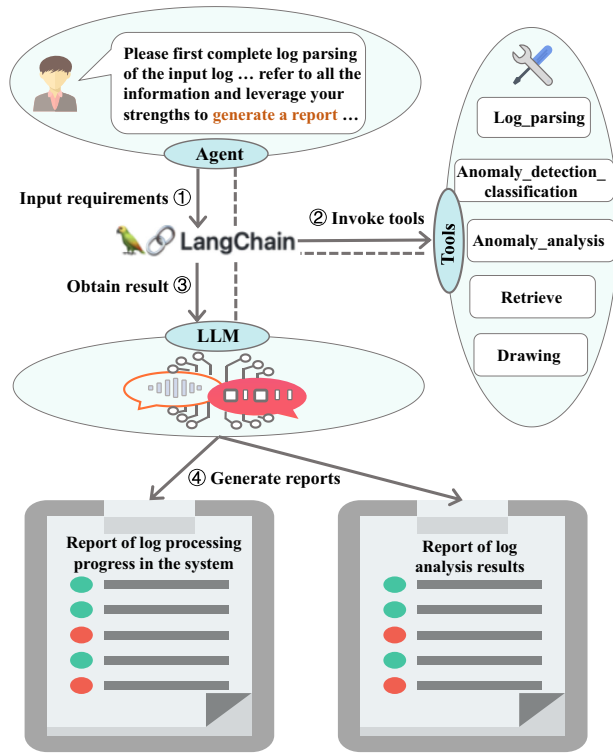


Fig. 4. Agents and tools of LangChain interact with the LLM to generate reports.

4. DEMONSTRATION

A. Datasets

The dataset of optical network logs used in this paper is composed of switch and optical transport network logs provided by an operator from 31 October 2023 to 2 November 2023. Introducing switch logs enables a more comprehensive analysis of optical networks. To make the fine-tuning dataset content more diverse and more suitable for anomaly detection and analysis, we have processed the collected data to include as many log types as possible, especially abnormal types. The processed logs were annotated by domain experts. They provided labels for log parsing, anomaly detection and classification, and anomaly analysis. The dataset used for log parsing, anomaly detection and classification, consists of 4975 records, divided into an 80% training set and a 20% testing set. The training set consists of 1914 normal logs and 2035 abnormal logs, while the testing set consists of 518 normal logs and 508 abnormal logs. Since anomaly analysis has different results for different templates of abnormal logs, one sample per abnormal log template was retained to construct the fine-tuning dataset for anomaly analysis. This dataset consists of 609 records, divided into a 90% training set and a 10% testing set.

B. Baselines

1. Log Parsing

ONLA-LLM is compared with the following baselines and GPT-4 in the log parsing task:

Drain [22]: Drain efficiently clusters log messages into groups with a parsing tree, streamlining the matching process based on message length and tokens for quick assignment.

Spell [23]: Spell employs an LCS-based method to efficiently group and update similar log entries, later enhanced with faster search, parallel processing, and semantic analysis.

AEL [39]: AEL is based on the clone detection technique and employs several heuristics to identify variable tokens for each log message and group messages with a similar number of tokens and parameters into bins.

Logram [25]: Logram uses an n -gram model to parse logs, reducing n -grams below a threshold to uncommon 2-grams and identifying variables through overlapping tokens.

GPT-4 [17]: GPT-4 is a large-scale multimodal model based on the Transformer framework. It captures the semantic and contextual information in logs to perform log anomaly detection and classification.

Few-shot learning [40]: Few-shot learning enables LLMs to learn from a limited number of labeled instances, typically just a few, and recognize new unseen instances. We utilize GPT-4 to implement few-shot learning. We utilize spaCy [41] to calculate text similarity and select the four labeled instances that are most similar to the queried log.

2. Anomaly Detection and Classification

ONLA-LLM is compared with the following baselines and GPT-4 in anomaly detection and classification:

Principal component analysis (PCA) [42]: PCA is an unsupervised machine learning algorithm. It represents the frequency of log key sequences by constructing a count matrix and then reduces the dimensionality of the original matrix to detect abnormal log sequences.

One-class SVM (OCSVM) [24]: OCSVM is a variant of SVM used for unsupervised log anomaly detection. It establishes a model using only normal data and identifies log sequences that deviate from the normal pattern.

LogAnomaly [26]: LogAnomaly is a framework based on LSTM networks that models log streams as natural language sequences. It can detect both sequential and quantitative anomalies in log sequences.

LogBERT [27]: LogBERT is a self-supervised framework based on BERT. It learns normal log sequence patterns through self-supervised training tasks and detects abnormal log sequences using prediction errors.

3. Anomaly Analysis

ONLA-LLM is compared with GPT-4 in anomaly analysis task.

C. Evaluation Metrics

1. Log Parsing

Exact match rate (EMR), precision accuracy (PA), and recall accuracy (RA) are used as evaluation metrics for log parsing.

EMR can be defined as the number of log entries that exactly matches the expert labels, divided by the total number of log entries. Its calculation formula is

$$EMR = FM/TL, \quad (3)$$

where FM represents the number of log entries with exact matches and TL represents the total number of log entries.

PA is defined as the sum of the ratios of correctly parsed parameters to the total parsed parameters for each log, divided by the total number of logs. Its calculation formula is

$$PA = \left[\sum (CP_i/TP_i) \right] / TL, \quad (4)$$

where CP represents the number of parameters correctly parsed in each log entry, and TP represents the total number of parsed parameters.

RA is defined as the sum of the ratios of correctly parsed parameters to the total number of parameters in the reference standard, divided by the total number of logs. It reflects the coverage capability. Its calculation formula is

$$RA = \left[\sum (CP_i/TA_i) \right] / TL, \quad (5)$$

where TA represents the total number of parameters in the reference standard of each log.

2. Anomaly Detection and Classification

- (1) Anomaly detection: Precision, Recall, and F1-score are used as evaluation metrics for anomaly detection. Among them, TP refers to the number of samples correctly detected as abnormal by the model, FP refers to the number of samples incorrectly detected as abnormal by the model, FN refers to the number of samples incorrectly detected as normal by the model, and TN refers to the number of samples correctly detected as normal by the model.

Precision measures the proportion of correctly identified anomaly samples (TP), out of all the anomalies detected by the model and is calculated as

$$\text{Precision} = TP / (TP + FP). \quad (6)$$

Recall measures the proportion of correctly identified anomaly samples (TP) out of all real anomalies and is calculated as

$$\text{Recall} = TP / (TP + FN). \quad (7)$$

F1-score is the harmonic mean of the Precision and Recall and is calculated as

$$\text{F1-score} = [2 * (\text{Precision} * \text{Recall})] / (\text{Precision} + \text{Recall}). \quad (8)$$

- (2) Anomaly classification: Due to the class imbalance in the anomaly classification dataset, weighted average precision, recall, and F1-score were introduced to better reflect the comprehensive ability of the log anomaly classification model. The calculation formula is as follows:

$$\text{Weighted-precision} = \sum \text{Precision}_i * W_i, \quad (9)$$

$$\text{Weight-recall} = \sum \text{Recall}_i * W_i, \quad (10)$$

$$\text{Weight-F1-score} = \sum \text{F1-score}_i * W_i. \quad (11)$$

Among them, Precision_i, Recall_i, and F1-score_i represent the precision, recall, and F1-score of class *i*, and *W_i* represents the proportion of class *i* in the dataset.

3. Anomaly Analysis

The anomaly analysis task is evaluated using three metrics: key text coverage, usability, and fluency. For each metric, two experts assign scores ranging from 1 to 10 based on definitions of metrics. The final score is obtained by averaging the scores given by the two experts.

Key text coverage: Key text coverage assesses the accuracy of the anomaly analysis results. It emphasizes whether the results cover the key content of the expert-annotated anomaly analysis labels.

Usability: Usability evaluates whether results contain the correct results clearly and concisely. It highlights that the anomaly analysis results generated by the model should be presented in a succinct manner, allowing users to grasp the key points quickly.

Fluency: Fluency refers to the coherence and naturalness of the text generated by the LLM.

D. Implementation Details

Since the chat version of LLaMA2 demonstrates better performance in instruction following and context learning [43], LLaMA2-7b-chat is used as our base model for instruction tuning. Our demonstration was conducted on an NVIDIA A40 48 GB GPU. We conducted repeated experiments to determine the optimal parameters. We set the attention dimension of LoRA to 16, the alpha parameter of LoRA scaling to 16, and apply a dropout probability of 0.05 to LoRA layers. For the hyperparameters of log parsing, we set the batch size to 1, the gradient accumulation steps to 2, and the learning rate to 1e-5. We trained the model for four epochs using the AdamW optimizer and a constant learning rate schedule. For the hyperparameters of anomaly detection and classification, we set the batch size to 1, the gradient accumulation steps to 2, and the learning rate to 1e-5. We trained the model for 11 epochs using the AdamW optimizer and a constant learning rate schedule. For the hyperparameters of anomaly analysis, we set the batch size to 1, the gradient accumulation steps to 2, and the learning rate to 1e-5. We trained the model for 10 epochs using the AdamW optimizer and a constant learning rate schedule.

5. RESULTS AND ANALYSIS

A. Results of Log Parsing

We intuitively represent the model's learning process during training on the log parsing task through a loss function plot, as shown in Fig. 5. This figure displays the loss values obtained by setting the logging steps to 2500. The plot visualizes the change in steps on the *x*-axis and the corresponding change in loss values on the *y*-axis. The steady improvement of the model

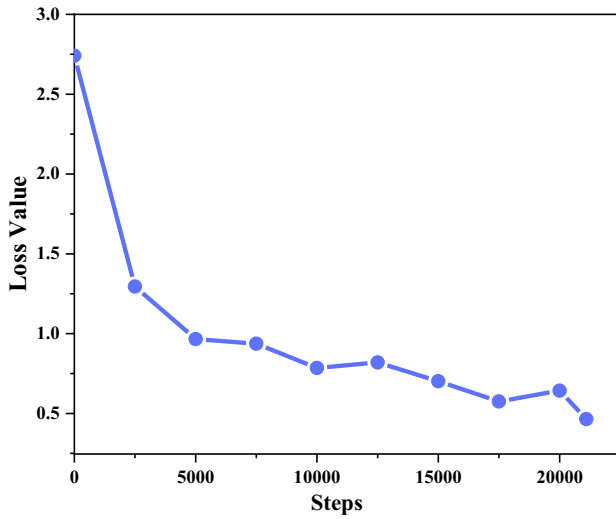


Fig. 5. The loss function plot during the training process of the model for log parsing task.

over time becomes immediately apparent, as indicated by the decreasing loss values.

Existing log parsers parse log information into templates, which is one of the functionalities achieved by ONLA-LLM. Therefore, we first compared the log template parsing capability of ONLA-LLM with four state-of-the-art log parsers, as shown in Table 1. The best results for each metric are highlighted in bold. The results indicate that existing log parsers achieve lower performance. Among them, AEL, Spell, and Drain all obtained EMR scores around 0.2, and PA and RA scores around 0.4. These three parsers are based on heuristic methods, which use different heuristic rules and tree structures to extract templates from logs. These three methods have limited robustness and low accuracy, primarily because they require encoding domain expertise into effective heuristic rules. Therefore, they often cannot perform well in logs that do not match the manually designed features. Since datasets in this paper are optical network logs, there are many domain-specific nouns and descriptions, as well as diverse log information. If rules are manually constructed for each type of log, it will be time-consuming and laborious, so it is not within our consideration. Logram gets the lowest EMR, PA, and RA scores because Logram is based on frequent pattern mining, which uses the mined frequent patterns as log templates. However, there are few repeated logs in the dataset of this paper, which makes it difficult to extract log templates and leads to low scores. ONLA-LLM benefits from fine-tuning techniques and a LLM base. It learns domain-specific nouns and descriptions through fine-tuning and improves language understanding and analytical capabilities through the LLM base. Consequently, it achieves significantly better performance than existing log parsers, with a EMR of 0.82, PA of 0.94, and RA of 0.95.

Zero-shot learning and few-shot learning based on GPT-4 were chosen as benchmarks for comparing the overall performance of the log parsing task, and the results are shown in Fig. 6. We used the instructions and inputs of ONLA-LLM as the prompt for GPT-4-based zero-shot learning to ensure

Table 1. Performance Results of Log Parsing Template

Method	Metric		
	EMR	PA	RA
AEL	0.23	0.41	0.36
Drain	0.25	0.49	0.42
Spell	0.22	0.42	0.38
Logram	0.10	0.23	0.19
ONLA-LLM	0.82	0.95	0.94

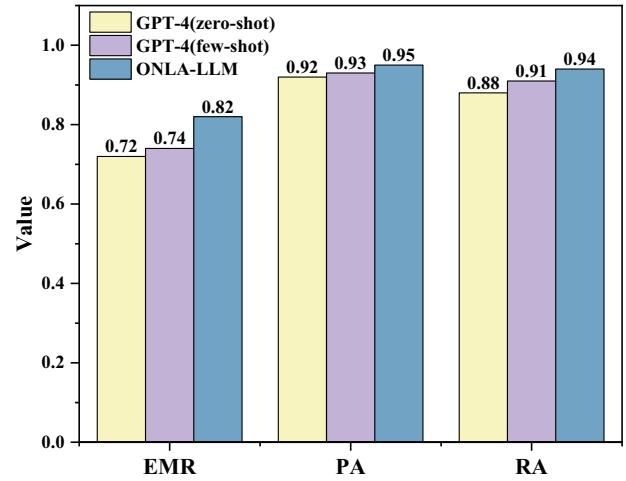


Fig. 6. Performance results of log parsing.

fairness. It can be observed that GPT-4-based zero-shot learning achieved good results. GPT-4-based few-shot learning achieves slight performance improvement over zero-shot learning because it learns from corresponding examples. But ONLA-LLM still improved by 11%, 2%, and 3% in EMR, PA, and RA, respectively, compared with GPT-4-based few-shot learning. The analysis found that GPT-4 cannot fully understand the technical terms of optical network logs (such as network elements and interface names), resulting in incomplete parameter parsing or failure to identify all variable parts. On the other hand, ONLA-LLM fine-tunes the LLM with data containing technical terms of optical network logs, making it more suitable for log parsing. An example of log analysis using ONLA-LLM is shown in Fig. 7.

B. Results of Anomaly Detection and Classification

We intuitively represent the model's learning process during training on the anomaly detection and classification task through a loss function plot, as shown in Fig. 8. This figure displays the loss values obtained by setting the logging steps to 2000. The decreasing loss values of these checkpoints emphasize the continuous improvement of the model. We compared six baselines for the anomaly detection task, including machine learning-based, deep learning-based, and semantic-based ones. The results are shown in Table 2, with the best results highlighted in bold. The results show that the machine learning-based methods PCA and OCSVM achieved the worst results. This could be due to information loss during the

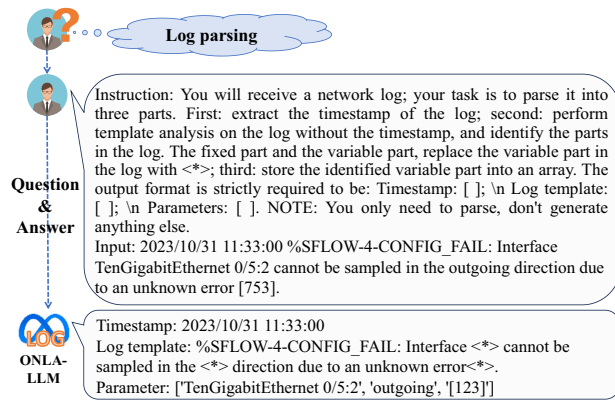


Fig. 7. An example of log parsing using ONLA-LLM.

dimensionality reduction process of PCA. OCSVM uses normal data to build its model, which means that the scarcity of normal logs in the dataset can make it challenging for OCSVM to generalize to anomalous logs. In terms of F1 score, the deep learning-based methods, LogAnomaly and LogBERT, achieved results second only to ONLA-LLM and outperformed GPT-4. This is because LogAnomaly and LogBERT are trained on log data from optical networks. LogAnomaly uses an LSTM network to capture the sequential and quantitative anomalies in the log sequences, while LogBERT uses a BERT model to learn the semantic and contextual information of the log sequences. LogAnomaly and LogBERT can be more targeted to the specific patterns and features in optical network logs, resulting in better anomaly pattern detection. In contrast, although GPT-4 is a powerful LLM, it is trained on a general text dataset and cannot be fine-tuned. Therefore, GPT-4-based zero-shot learning has difficulty in generalizing to the specific anomalous patterns of optical network logs, making it less specialized and effective than LogAnomaly and LogBERT. While GPT-4-based few-shot learning can capture semantic information better than zero-shot learning by learning from a few examples, its performance results, especially in terms of F1-score, are still lower than the fine-tuning approach we propose. Our proposed instruction tuned ONLA-LLM achieved the optimal precision of 0.97 and the optimal F1-score of 0.97 and obtained the suboptimal recall of 0.98. This is because LLaMA2 has the powerful language understanding capabilities. LLaMA2 can better adapt to this downstream task when instruction is tuned specifically for log anomaly detection tasks. ONLA-LLM achieves good results in terms of precision, recall, and F1-score, which indicates that ONLA-LLM has strong anomaly detection capabilities and accuracy and is a promising approach for anomaly detection of optical network logs. The optical network logs mentioned in this paper mainly need to identify errors and warning information, which do not rely on contextual information. However, in the future, it is worth considering using fine-tuning to further improve the anomaly detection in optical network logs that requires context information, so as to make the anomaly detection more complete.

Zero-shot learning and few-shot learning based on GPT-4 were chosen as benchmarks for comparison for the anomaly classification task. We used the instructions and inputs of

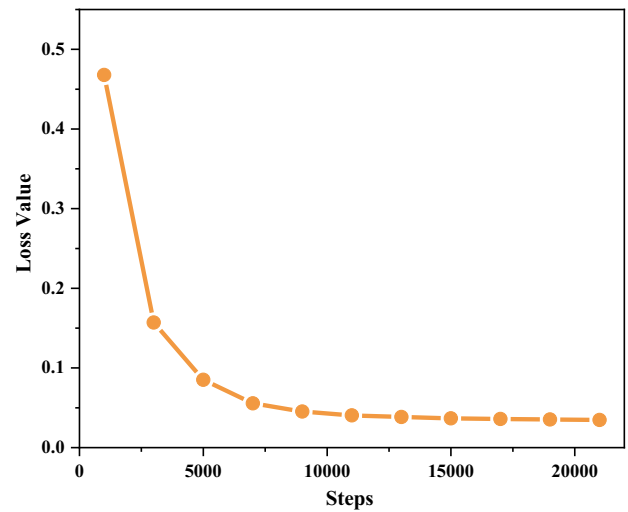


Fig. 8. The loss function plot during the training process of the model for anomaly detection and classification task.

Table 2. Performance Results of Anomaly Detection

Method	Metric		
	Precision	Recall	F1-Score
PCA	0.70	0.60	0.65
OCSVM	0.29	0.36	0.32
LogAnomaly	0.75	1.00	0.86
LogBERT	0.87	0.86	0.86
GPT-4 (zero-shot)	0.64	0.98	0.78
GPT-4 (few-shot)	0.81	0.93	0.86
ONLA-LLM	0.97	0.98	0.97

ONLA-LLM as the prompt for GPT-4-based zero-shot learning to ensure fairness. The results are shown in Fig. 9. ONLA-LLM achieved the highest precision, recall, and F1-score. The poor performance of GPT-4-based zero-shot learning can be attributed to two main reasons. First, when the words present in the logs overlap with the words in the anomaly class names, GPT-4 may tend to simply classify the logs into that specific class without considering the underlying context and deeper background. Second, the classification of optical network anomaly logs varies depending on the needs of different systems. GPT-4 is a general language model that has not been specifically trained for a particular domain, making it unable to accurately capture the subtle differences related to network anomalies. GPT-4-based few-shot learning's performance improves after providing some examples for learning. This is because GPT-4 learns the requirements of classification from similar examples. However, it provides limited generalization performance due to the limited number of examples. On the other hand, ONLA-LLM can learn the classification method of this network system through instruction tuning, which makes it more effective to distinguish between different anomaly categories. In addition, ONLA-LLM can also perform anomaly detection and classification tasks together. An example of anomaly detection and classification using ONLA-LLM is shown in Fig. 10. ONLA-LLM can correctly identify the log as "abnormal" and classify it as "network protocols and services."

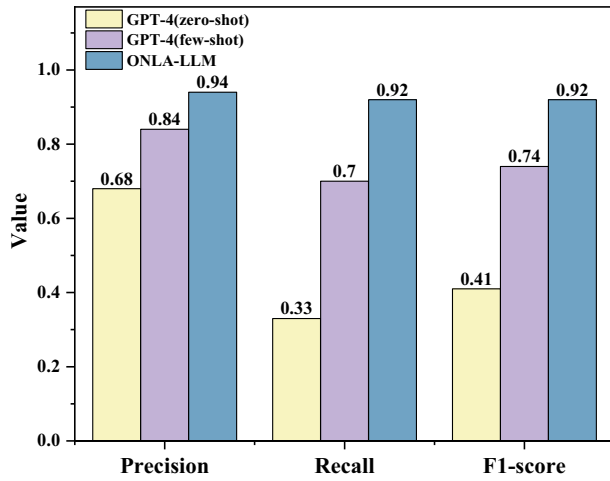


Fig. 9. Performance results of anomaly classification.

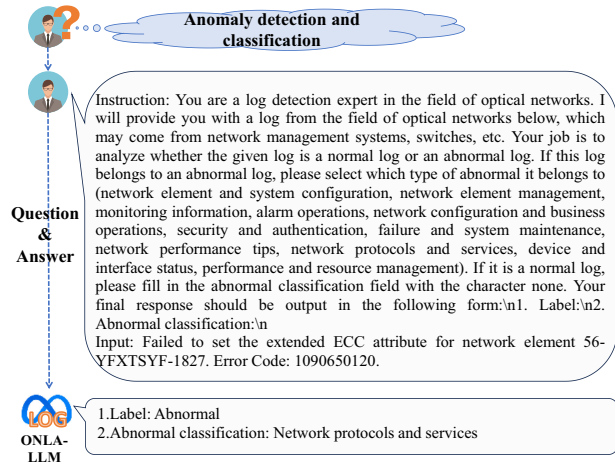


Fig. 10. An example of anomaly detection and classification using ONLA-LLM.

C. Results of Anomaly Analysis

We intuitively represent the model's learning process during training on the anomaly analysis task through a loss function plot, as shown in Fig. 11. This figure displays the loss values obtained by setting the logging steps to 250. For the anomaly analysis task, GPT-4-based zero-shot learning and few-shot learning are selected as the baseline for comparison, and the results are shown in Fig. 12. The best results for each metric are highlighted in bold. It can be observed that ONLA-LLM achieves a key text coverage of 8.67. According to our observations, ONLA-LLM provides meaningful and informative results in most cases, which provide valuable references for network administrators. However, there are approximately four instances in the test set where the suggested actions provided by ONLA-LLM are fewer than the expert labels. This leads to the inability to provide optimal recommendations in some cases, thus lowering the overall accuracy. On the other hand, GPT-4-based methods achieve higher key text coverage. The output of GPT-4 is verbose and contains a wealth of information. Therefore, it has a higher likelihood to cover the content of expert labels.

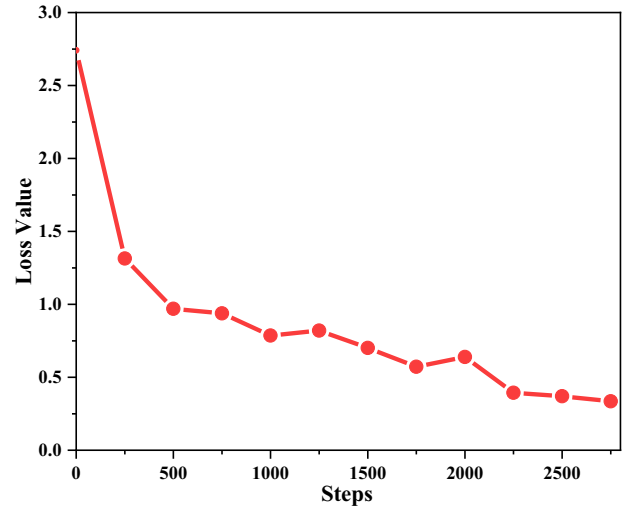


Fig. 11. The loss function plot during the training process of the model for the anomaly analysis task.

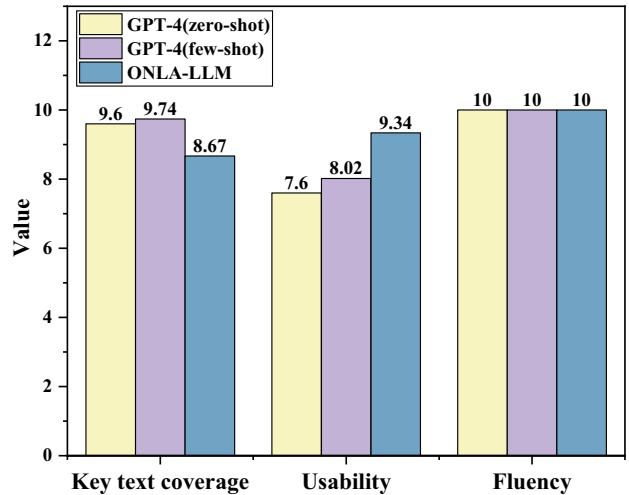


Fig. 12. Performance results of the anomaly analysis.

Among the three metrics, usability is a metric that balances accuracy and conciseness and can better indicate the guiding effect of the answer. ONLA-LLM achieves a higher usability of 9.34. ONLA-LLM's output is concise and clear, which contains accurate and usable results. Such outcomes help network administrators quickly obtain valuable information, thereby improving the efficiency of anomaly analysis. In contrast, GPT-4's usability is much lower than ONLA-LLM's. ONLA-LLM's answers consist of approximately 50 to 100 words, while GPT-4's answers range from 700 to 1000 words. This is because ONLA-LLM can learn language patterns, structures, and word counts that adapt to our needs through instruction tuning. GPT's answers provide a lot of background information that is not relevant to solving the problem. In addition, GPT-4's answers are full of clichés, such as "check the hardware," "check if the configuration is correct," "check the network connection," and "contact customer support/vendor for further solutions." Furthermore, there is a lot of repetitive content in the answers and the steps are complicated. For example, the

answer gives three subheadings: “Possible Causes,” “Diagnostic Steps,” and “Solutions.” But the content is highly similar in fact. Its core content can be summarized in one sentence. GPT-4’s responses tend to be circuitous, lacking a direct focus on key points, making it less convenient for network administrators to quickly grasp essential information, thus resulting in lower usability. GPT-4-based few-shot learning shows a slight improvement in usability compared to zero-shot learning. Its responses are between 200–300 words. The answers generated through few-shot learning are more concise compared to zero-shot learning, and they are structured into two parts: root cause analysis and processing suggestions. This indicates that GPT-4 has learned the format and preferences from the provided examples.

Both ONLA-LLM and GPT-4-based methods generate highly fluent text. As proficient LLMs, they ensure coherence, naturalness, and readability in the generated text. In conclusion, although GPT-4’s answers have higher accuracy, this comes at the cost of verbosity and complexity. Network administrators require concise and straightforward anomaly analysis results, hence the results output by ONLA-LLM are more usable. ONLA-LLM can provide clear guidance and avoid unnecessary operations.

Two examples of anomaly analysis are demonstrated by using ONLA-LLM, as shown in Fig. 13. In Fig. 13 log A, ONLA-LLM analyzes an anomalous template and identifies that the “exception originates from E2E management.” It suggests checking for network connectivity issues and hardware failures at both ends. Network administrators can quickly understand the network anomaly and take actions promptly based on ONLA-LLM’s analysis. They can determine if there are any disconnected, unstable, or abnormal connections by checking for network connectivity issues. They can repair or replace faulty hardware by inspecting the hardware devices at both ends. ONLA-LLM’s detection and analysis of this abnormal log helps expedite the troubleshooting process, leading to a faster recovery of normal network operations and ensuring business continuity and stability. The anomaly in Fig. 13 log B is mainly caused by “insufficient hardware forwarding plane (FP) resources,” and the suggested treatment is to “consider removing rules that are no longer needed or upgrading devices.” Addressing this anomaly promptly can prevent issues such as packet loss, service interruptions, and degraded network performance when the FP capacity is insufficient to handle all incoming packets. ONLA-LLM guides network

administrators to discover FP bottlenecks in their networks. They can reduce latency and packet loss and perform timely network optimization by freeing up FP resources.

D. Report Generation

The agent in LangChain autonomously determines the next action after receiving a task, which is achieved through the ReAct framework in the Agent class. The ReAct framework prompts the LLMs to generate reasoning trajectories and actions for the task, following a pattern of thought, action, and observation, as shown in Figs. 14 and 15. In the “Reasoning” phase, the model observes the current environment and state and generates a reasoning trajectory. In the “Action” phase, the model takes the next step, such as interacting with external sources like knowledge repositories or tools to gather information and provide the final answer.

The log analysis report is shown in Fig. 14. After receiving the requirements from the network administrator, ONLA-LLM analyzes and determines the next action. First, ONLA-LLM performs log parsing by invoking the log parsing tool and presents the parsing results. Then, ONLA-LLM reasons about the need for anomaly detection and classification after log parsing. Once anomaly detection and classification are completed, ONLA-LLM utilizes the anomaly analysis tool for further analysis. When the above processes are completed, ONLA-LLM generates a final report containing the results of the above steps. The progress report of system log processing is shown in Fig. 15. When the network administrator requests information on the progress of log processing and visualized anomaly classification results, ONLA-LLM first utilizes the “Retriever” tool to retrieve relevant data about the system log analysis process. Subsequently, it employs the “Drawing” tool to visualize the anomaly classification results. Finally, it generates a comprehensive report encompassing the aforementioned information, with a final summary in percentage. Among them, the model used for generating summaries is GPT-4. This is because larger general models like GPT series are ideal as foundational models for information extraction and integration. These large-scale general models serve as the “brains,” while task-specific fine-tuned models and various tools act as the “arms.” This combination can enhance performance.

Integrating LLMs with external tools and ReAct empowers intelligent interaction for network operations. Automating

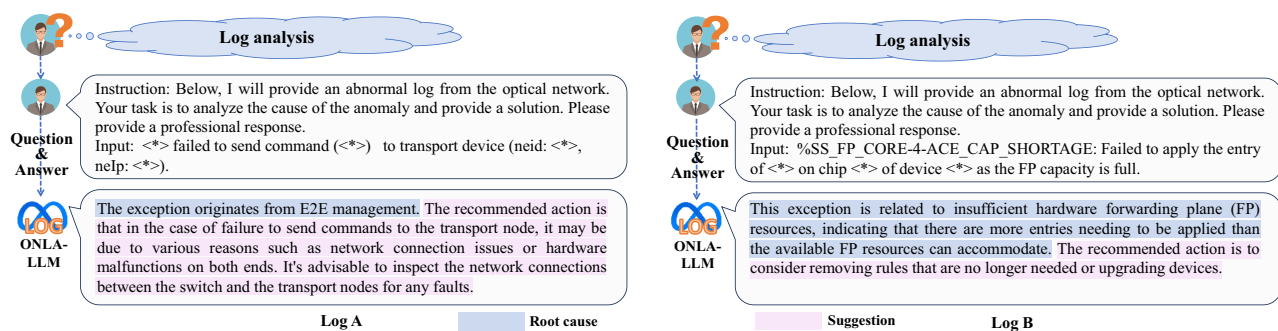


Fig. 13. Two examples of anomaly analysis using ONLA-LLM. Log A provides analysis and resolution recommendations regarding E2E management anomalies. Log B focuses on the analysis and resolution recommendations for insufficient hardware forwarding plane resources.

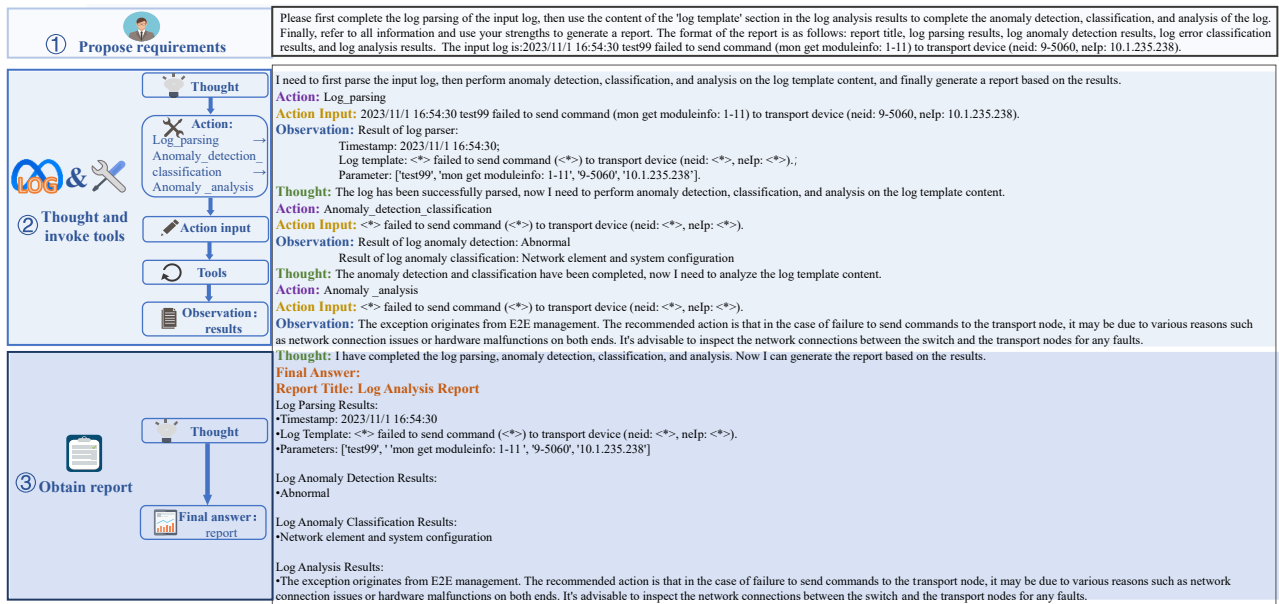


Fig. 14. Report of log processing progress in the system. When a network administrator inputs a requirement, the agent of LangChain autonomously determines the next actions. It invokes tools and obtains results, and finally integrates all the results into a report.

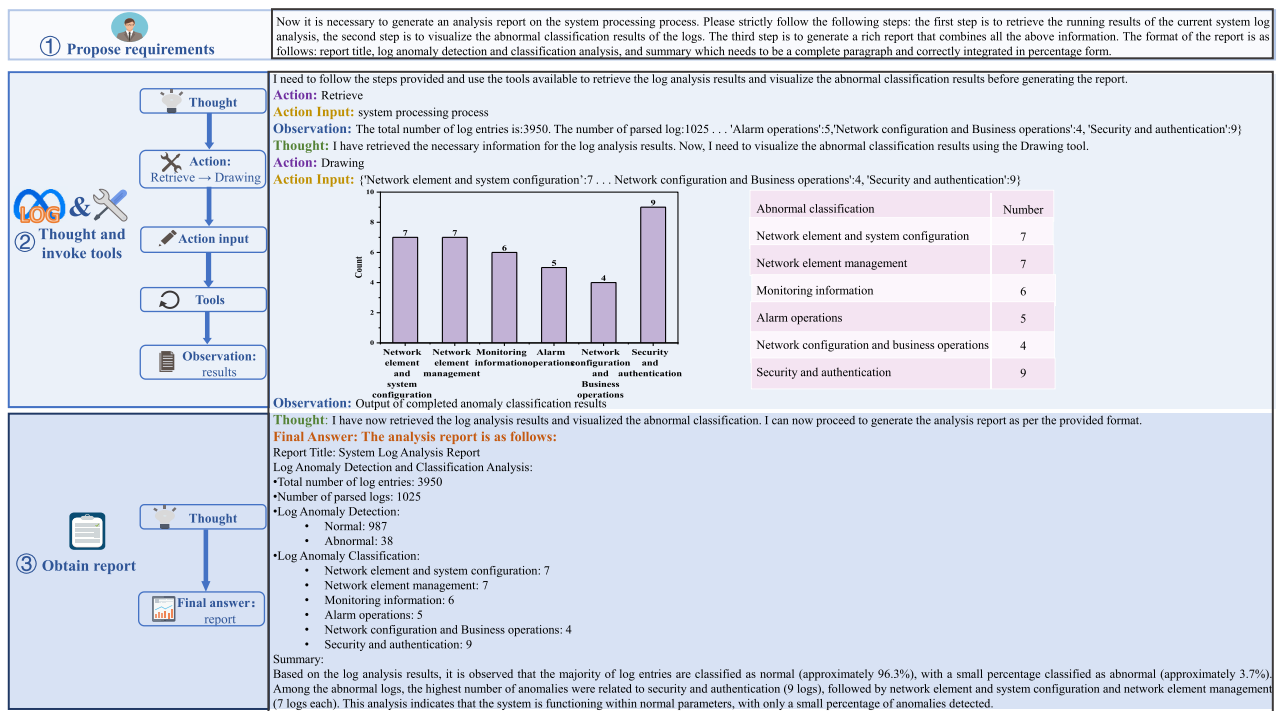


Fig. 15. Report of log analysis results. When a network administrator inputs a requirement, the agent of LangChain autonomously determines the next actions. It invokes tools and obtains results, and finally integrates all the results into a report.

reasoning and invoking external supporting tools simplifies data processing and enhances result presentation. Network operators can benefit from a more intelligent interactive experience, reducing the burden of manual operations and improving work efficiency. This integration can assist network operations, providing more efficient and intelligent support, and promoting the automation of network management. We utilize these two examples to showcase the capabilities of LangChain

as an agent, which can integrate with other tools and services to automate the execution of more complex tasks. In the future, various machine learning and deep learning models can also be defined as tools as requirements change. Ultimately, LangChain accesses and processes information, automates tasks, invokes tools, and combines information from different sources to generate structured and visual reports.

E. Comparison of Fine-Tuned and Non-Fine-Tuned Models

1. Log Parsing

We compared the log parsing performance of ONLA-LLM with LLaMA2-7b-chat, which was not fine-tuned, and the results are shown in Fig. 16. It can be observed that ONLA-LLM is far superior to LLaMA2-7b-chat in the three metrics EMR, PA, and RA. We analyzed the error outputs from LLaMA2-7b-chat. The first reason for the error is that (*) is not used to replace the variable part as required, but instead replaced with words or other symbols, as shown in log “A” in Fig. 17. The second is that the timestamp format is changed during the parsing process, as shown in log “A” in Fig. 17. The third is that some content is lost during the parsing into the template, as shown in log “A” in Fig. 17. The fourth is that the fixed part and variable part in the log are not parsed correctly, as shown in “B” in Fig. 17. These errors are mainly due to the fact that LLaMA2-7b-chat has a smaller model size, which results in weaker language understanding capabilities. LLaMA2-7b-chat cannot be parsed well due to the lack of background knowledge when faced with optical network data. On the other hand, ONLA-LLM introduces background knowledge through instruction tuning and standardizes the output format, thus obtaining better results.

2. Anomaly Detection and Classification

We compared the anomaly detection performance of ONLA-LLM with LLaMA2-7b-chat, which is not fine-tuned, and the results are shown in Fig. 18. It can be observed that ONLA-LLM significantly outperforms LLaMA2-7b-chat in all three metrics. Among them, the precision is improved by 0.26, the recall is improved by 0.91, and the F1-score is improved by 0.85. Similarly, the performance comparison results of anomaly classification are shown in Fig. 19, and the results of ONLA-LLM are still much higher than those of LLaMA2-7b-chat. Among them, the precision is improved by 0.68, the recall is improved by 0.45, and the F1-score is improved by 0.57. We selected an example shown in Fig. 20 to illustrate the improvement after instruction tuning. The log in the figure shows that the performance value of the RMON module exceeds the preset upper limit. ONLA-LLM, leveraging the background knowledge of optical networks, can identify that the purpose of this log message is to alert the network administrator of a potential performance issue or anomaly. However, LLaMA2-7b-chat without instruction tuning cannot understand the deeper meaning of this log. Therefore, instruction

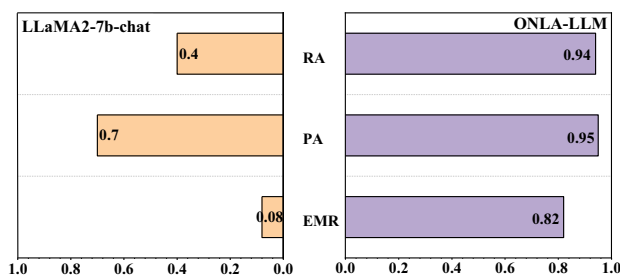


Fig. 16. Performance results of log parsing between ONLA-LLM and LLaMA2-7b-chat.

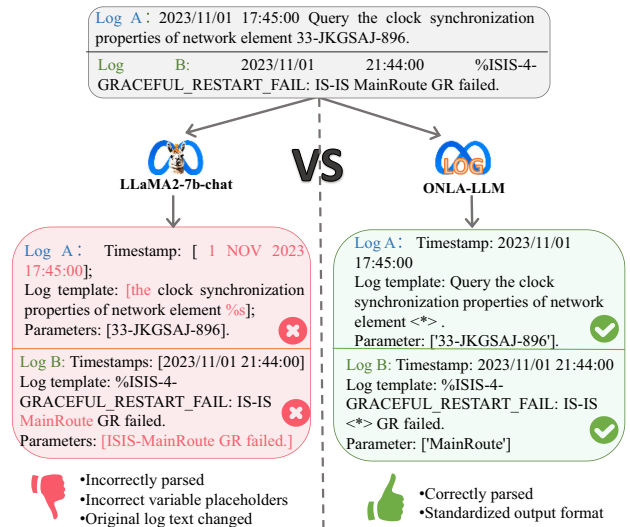


Fig. 17. Comparison of log parsing results between ONLA-LLM and LLaMA2-7b-chat.

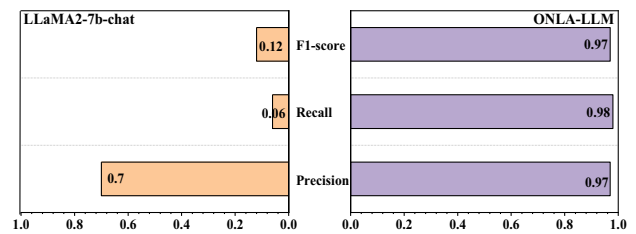


Fig. 18. Performance results of anomaly detection between ONLA-LLM and LLaMA2-7b-chat.

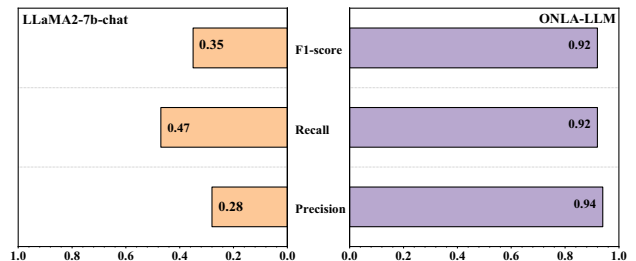


Fig. 19. Performance results of anomaly classification between ONLA-LLM and LLaMA2-7b-chat.

tuning can incorporate the knowledge and experience of optical network professionals into the model, enabling it to better understand optical network anomalies and improve sensitivity and accuracy in detecting optical network anomalies.

3. Anomaly Analysis

We compared the anomaly analysis performance of ONLA-LLM with LLaMA2-7b-chat, and the results are shown in Fig. 21. In addition to comparable fluency, ONLA-LLM outperforms LLaMA2-7b-chat in terms of key text coverage and usability. Analysis reveals that LLaMA2-7b-chat provided numerous incorrect or irrelevant answers. It frequently generates uncertain phrases such as “seems like” or “possibly.”

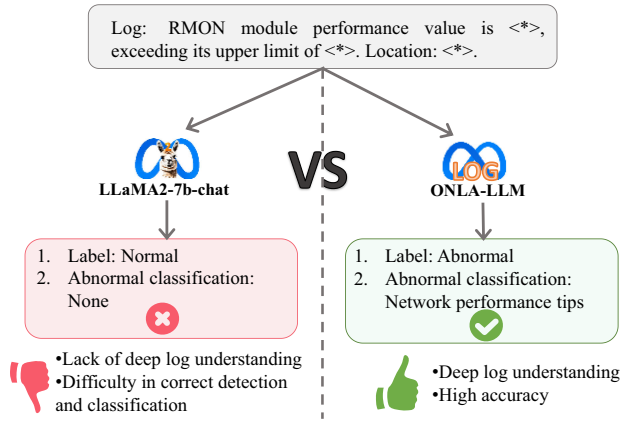


Fig. 20. Comparison of anomaly detection and classification results between ONLA-LLM and LLaMA2-7b-chat.

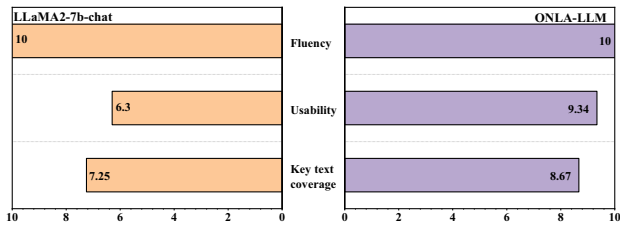


Fig. 21. Performance results of anomaly analysis between ONLA-LLM and LLaMA2-7b-chat.

Furthermore, the responses from LLaMA2-7b-chat are about 300–500 words, which is 5 times longer than ONLA-LLM's responses. These verbose answers contain many unnecessary processing suggestions, complicating simple problems. There are also problems such as indirect answers and serious clichés. These issues result in lower key text coverage and usability for LLaMA2-7b-chat. On the other hand, ONLA-LLM has learned the language style, standardized the answer format, and word count of anomaly analysis through instruction tuning. This is because, when constructing the fine-tuning dataset, labeled text align with the style and length requirements. At the same time, instruction tuning also injects more background knowledge into ONLA-LLM, improving its accuracy and usability. Figure 22 shows a comparison of anomaly analysis between ONLA-LLM and LLaMA2-7b-chat. The output of LLaMA2-7b-chat is incorrect, verbose, and full of clichés, while the output of ONLA-LLM is concise and accurate.

F. Comparison of Separate Fine-Tuning and Combined Fine-Tuning

We compared the advantages and disadvantages of separate fine-tuning and combined fine-tuning by consolidating the three fine-tuning processes into one. The test results are shown in Table 3. The performance metrics for each task have decreased compared to separate fine-tuning. Some metrics, such as EMR for log parsing, F1-score for anomaly detection and classification, and usability for anomaly analysis, have experienced significant decreases, with reductions of 18%, 15%, 35%, and 8% respectively. This is because combined fine-tuning may lead to task conflicts and interference, where

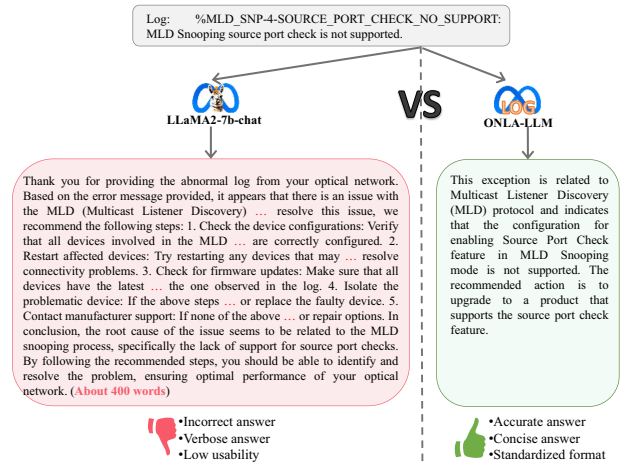


Fig. 22. Comparison of anomaly analysis results between DCNLA-LLM and LLaMA2-7b-chat.

certain features perform well on one task but have negative effects on another. Separate fine-tuning allows each model to focus on its task's objectives, thereby facilitating better optimization and performance. In addition, separate fine-tuning has greater flexibility and adaptability, allowing specific task requirements to be independently retrained without affecting other models. This modular characteristic proves particularly beneficial in evolving systems and changing environments. The widely adopted mixture-of-experts (MoE) architecture in LLMs follows a similar approach to enhance the accuracy and quality of LLM outputs [44]. However, it is important to note that combined fine-tuning may also work well in some cases, especially when tasks exhibit strong correlation or similarity. In addition, combined fine-tuning can complete all tasks with only one model, saving storage and computing resources, but it comes at the cost of accuracy. The best fine-tuning strategy depends on the specific task and data characteristics, so experimentation and evaluation are needed in practice to determine the most appropriate approach.

G. Response Time

To gain a more detailed understanding of the response time of the proposed method, we conducted tests on the response time of ONLA-LLM and benchmark methods in log parsing, anomaly detection and classification, and anomaly analysis. We conducted multiple experiments and took the average. For log parsing, the response time of ONLA-LLM is about 5.4 s, while the response time of GPT-4-based zero-shot learning and GPT-4-based few-shot learning is about 3.4 s and 3.1 s, respectively. The response time of other methods is far less than 1 s. For anomaly detection and classification, the response time of ONLA-LLM is about 2.2 s, while the response time of GPT-4-based zero-shot learning and GPT-4-based few-shot learning is about 1.51 s and 1.48 s, respectively. The response time of other methods is far less than 1 s. For anomaly analysis, the response time of ONLA-LLM is about 14.1 s, while the response time of GPT-4-based zero-shot learning and GPT-4-based few-shot learning is about 10.9 s and 6.7 s, respectively.

Table 3. Performance Comparison of Combined and Separate Fine-Tuning

Log Parsing			Anomaly Detection			Anomaly Classification			Anomaly Analysis		
Metric	Combined Fine-tuning	Separate Fine-tuning	Metric	Combined Fine-tuning	Separate Fine-tuning	Metric	Combined Fine-tuning	Separate Fine-tuning	Metric	Combined Fine-tuning	Separate Fine-tuning
EMR	0.67	0.82	Precision	0.77	0.97	Precision	0.67	0.94	Key text coverage	8.47	8.67
PA	0.92	0.95	Recall	0.89	0.98	Recall	0.59	0.92	Usability	8.58	9.34
RA	0.86	0.94	F1-score	0.82	0.97	F1-score	0.6	0.92	Fluency	10	10

The response time of LLM-based methods is much higher than that of baseline models. This is because LLMs adopt transformer-based architectures, have larger model sizes and parameters, and use an autoregressive approach to generate text. All of these lead to higher computational complexity. ONLA-LLM has the shortest response time for anomaly detection because its output content is the shortest compared to log parsing and anomaly analysis. The response time of GPT-4-based zero-shot learning is higher than that of GPT-4-based few-shot learning. This is because GPT-4-based zero-shot learning generates some redundant explanations after answering the required content. GPT-4-based few-shot learning learns the way of answering from the examples, and the model generates more concise responses, thus reducing the length of the generated text and shortening the response time. GPT-4-based methods have faster response speeds than ONLA-LLM because OpenAI online models use various inference acceleration methods. When ONLA-LLM is accelerated, its response time will also be greatly reduced. Although LLMs have higher computational complexity due to their size and complex architecture, they have significant advantages in handling complex tasks and generating creative text. Researchers are actively exploring ways to accelerate LLM inference, such as model compression, hardware acceleration, and algorithm optimization [45]. These efforts will significantly improve the speed and efficiency of LLM inference in the future.

6. LIMITATIONS AND FUTURE DIRECTIONS

We highlight some limitations of this paper and future research directions based on our findings. We hope that these analyses and the provided overview can further facilitate the application of fine-tuning LLMs in optical networks.

A. Model Size

Previous research has shown that the quality of the base model is crucial for downstream performance [46]. Models pretrained on larger (or potentially higher-quality) corpora are better suited as base models for fine-tuning. Due to limitations in computational resources, this paper only fine-tuned a LLaMA-2-7b model using the optical network logs. Smaller-sized models may not be able to fully capture the complex patterns, features, and contextual relationships in optical network logs, which could limit the model's performance. A direction for future work is to demonstrate our methods on larger pretrained models, such as 13B or 70B parameters. Larger models possess a greater parameter scale, deeper network structures, and

stronger representation capabilities, which would enable them to achieve higher accuracy and performance in various tasks related to optical network log analysis.

B. Datasets

In future research, increasing the quantity and quality of the dataset has the potential to enhance the performance of fine-tuned LLMs [47]. Increasing the quantity and diversity of optical network log data can help LLMs learn more comprehensive patterns of anomalies, thereby improving their robustness and adaptability. Moreover, high-quality datasets are crucial for training LLMs. The accuracy of the annotation is highly relevant to the quality of the data. Ensuring that the annotation of the dataset samples is accurate and meets the requirements can improve the training quality and generalization ability of the model. This paper currently adopts the form of expert annotation, which is not very friendly to the cost of manpower and time. In the future, we should explore the use of information such as optical network system work orders, knowledge bases, and historical records, and adopt a combination of machine and manual methods to improve the quality and efficiency of annotation.

C. Evaluation Method

We employ open-ended evaluation metrics for the generative task of anomaly analysis in this paper. The evaluation method by industry experts is highly subjective. In the future, it is necessary to provide more comprehensive evaluation methods for optical network log analysis tasks. The most relevant evaluation metrics should be determined based on specific task requirements and feedback from domain experts. Consideration should be given to developing automated evaluation tools and protocols to reduce subjectivity and workload of human scoring. Exploring the use of LLMs to assist in evaluating model quality is also worth investigating. Ultimately, an evaluation method that meets the requirements of the field, comprehensive, and more automated should be constructed.

D. Hallucination

Hallucination in LLMs refers to the generation of unfounded or incorrect content in the model's output [48]. During the testing of the anomaly analysis task, one of the analysis results contained hallucination, as shown in Fig. 23. The log indicates that a network device failed to create a data collection instance. It requires checking the device connection, verifying the device

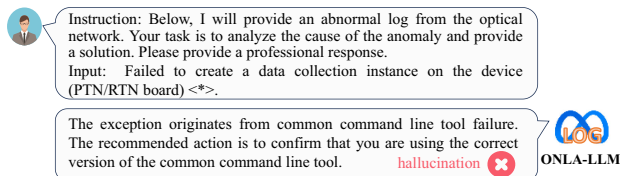


Fig. 23. An example of an anomaly analysis result that included hallucination.

compatibility, and examining the configuration information. However, the model incorrectly suggests verifying whether the version of the command line tool being used is correct. Such incorrect results could mislead subsequent judgments and operations. Therefore, hallucination is a hurdle in the development and application of LLMs. This may result in the output of the model being of only auxiliary use to the network administrators. Mitigating hallucination in LLMs calls for various approaches, such as fine-tuning with reinforcement learning from human feedback.

E. Applicable to an Optical Network

This work represents the first step in applying fine-tuned LLMs to optical networks. In addition to the functionalities mentioned in the paper, fine-tuned LLMs can be applied to other aspects of optical network log analysis. For instance, log summarization can be performed on log information over a period of time to generate a summary, which allows network administrators to quickly understand network trends. Since our approach is generic, it can be further applied to address other challenges in optical networks. For example, converting expertise and practices of optical network operations into knowledge for the model to learn, so that the model can answer questions from network management personnel and provide operational guidance. LLMs can also learn from information such as the state of the optical network, maintenance records, etc., to provide suggestions for network optimization and equipment maintenance [49]. In the future, it is possible to improve operational efficiency, reduce the risk of failures, and provide support for continuous improvement of network performance by combining the fine-tuned LLMs with the expertise and practices of optical network operations [50].

Furthermore, it is crucial to consider the requirements and obstacles of applying LLMs to real-world optical networks. Future research should explore optimization techniques for LLMs, such as quantization, improved self-attention mechanisms (e.g., PagedAttention, FlashAttention), and distributed inference frameworks (e.g., Megatron, DeepSpeed). These advances can improve the training and inference efficiency of LLMs and reduce costs, which will further promote the application of LLMs in optical networks.

7. CONCLUSION

This paper presents a LLM-based optical network log analysis method for network operation and maintenance with LLaMA2 using instruction tuning. The proposed method was comprehensively tested and analyzed for three tasks: log parsing, anomaly detection and classification, and anomaly

analysis. The results show that ONLA-LLM outperforms the baseline in all three tasks. ONLA-LLM is of great significance for optical network log analysis. However, ONLA-LLM still has some limitations, and it can be improved in various aspects of the scheme through various technologies in the future. We expect this paper to inspire thinking on how LLMs can promote the efficiency, accuracy, and automation of optical network operation and maintenance, thereby driving the further development of networks.

Funding. National Natural Science Foundation of China (62171053, U23B2008); Beijing Nova Program (20230484331); BUPT Excellent Ph.D. Students Foundation (CX2022117).

Acknowledgment. We would like to acknowledge China Mobile for sharing the valuable network log data and providing professional guidance on data processing and analysis.

Disclosures. The authors declare no conflicts of interest.

REFERENCES

1. K. Christodoulopoulos, C. Delezoide, N. Sambo, *et al.*, "Toward efficient, reliable, and autonomous optical networks: the ORCHESTRA solution," *J. Opt. Commun. Netw.* **11**, C10–C24 (2019).
2. X. Liu, H. Lun, R. Gao, *et al.*, "A data-fusion-assisted telemetry layer for autonomous optical networks," *J. Lightwave Technol.* **39**, 3400–3411 (2021).
3. L. Velasco, A. C. Piat, O. Gonzalez, *et al.*, "Monitoring and data analytics for optical networking: benefits, architectures, and use cases," *IEEE Netw.* **33**, 100–108 (2019).
4. M. Furdek, C. Natalino, F. Lipp, *et al.*, "Machine learning for optical network security monitoring: a practical perspective," *J. Lightwave Technol.* **38**, 2860–2871 (2020).
5. F. Musumeci, C. Rottondi, G. Corani, *et al.*, "A tutorial on machine learning for failure management in optical networks," *J. Lightwave Technol.* **37**, 4125–4139 (2019).
6. X. Chen, C. Y. Liu, R. Proietti, *et al.*, "Automating optical network fault management with machine learning," *IEEE Commun. Mag.* **60**(12), 88–94 (2022).
7. T. Tanaka, T. Inui, S. Kawai, *et al.*, "Monitoring and diagnostic technologies using deep neural networks for predictive optical network maintenance," *J. Opt. Commun. Netw.* **13**, E13–E22 (2021).
8. J. Mata, I. De Miguel, R. J. Durán, *et al.*, "Artificial intelligence (AI) methods in optical networks: a comprehensive survey," *Opt. Switch. Netw.* **28**, 43–57 (2018).
9. X. Liu, H. Lun, M. Fu, *et al.*, "AI-based modeling and monitoring techniques for future intelligent elastic optical networks," *Appl. Sci.* **10**, 363 (2020).
10. H. Singh, D. Ramya, R. Saravanakumar, *et al.*, "Artificial intelligence based quality of transmission predictive model for cognitive optical networks," *Optik* **257**, 168789 (2022).
11. K. Abdelli, J. Y. Cho, F. Azendorf, *et al.*, "Machine-learning-based anomaly detection in optical fiber monitoring," *J. Opt. Commun. Netw.* **14**, 365–375 (2022).
12. X. Chen, B. Li, R. Proietti, *et al.*, "Self-taught anomaly detection with hybrid unsupervised/supervised machine learning in optical networks," *J. Lightwave Technol.* **37**, 1742–1749 (2019).
13. D. Rafique, T. Szyrkowiec, H. Griebner, *et al.*, "Cognitive assurance architecture for optical network fault management," *J. Lightwave Technol.* **36**, 1443–1450 (2017).
14. P. Iovanna, M. Puleri, G. Bottari, *et al.*, "Intent-based AI system in packet-optical networks towards 6G," *J. Opt. Commun. Netw.* **16**, C31–C42 (2024).
15. C. Natalino, A. Udalcovs, L. Wosinska, *et al.*, "Spectrum anomaly detection for optical network monitoring using deep unsupervised learning," *IEEE Commun. Lett.* **25**, 1583–1586 (2021).
16. ChatGPT, <https://openai.com/blog/chatgpt>.
17. J. Achiam, S. Adler, S. Agarwal, *et al.*, "GPT-4 technical report," *arXiv* (2023).

18. M. U. Hadi, R. Qureshi, A. Shah, *et al.*, "A survey on large language models: applications, challenges, limitations, and practical usage," *Authorea* (2023).
19. Y. Wang, J. Li, Y. Pang, *et al.*, "AlarmGPT: an intelligent operation assistant for optical network alarm analysis using ChatGPT," in *49th European Conference on Optical Communications (ECOC)* (2023), pp. 1330–1333.
20. Y. Zhang, M. Zhang, Y. Song, *et al.*, "GPT-enabled digital twin assistant for multi-task cooperative management in autonomous optical network," in *Optical Fiber Communication Conference* (2023).
21. H. Touvron, L. Martin, K. Stone, *et al.*, "LLaMA 2: open foundation and fine-tuned chat models," *arXiv* (2023).
22. P. He, J. Zhu, Z. Zheng, *et al.*, "Drain: an online log parsing approach with fixed depth tree," in *24th International Conference on Web Services (ICWS)* (2017), pp. 33–40.
23. M. Du and F. Li, "Spell: streaming parsing of system event logs," in *IEEE 16th International Conference on Data Mining (ICDM)* (2016), pp. 859–864.
24. B. Schölkopf, J. C. Platt, J. Shawe-Taylor, *et al.*, "Estimating the support of a high-dimensional distribution," *Neural Comput.* **13**, 1443–1471 (2001).
25. H. Dai, H. Li, C. S. Chen, *et al.*, "Logram: efficient log parsing using n -gram dictionaries," *IEEE Trans. Softw. Eng.* **48**, 879–892 (2020).
26. W. Meng, Y. Liu, Y. Zhu, *et al.*, "LogAnomaly: unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *28th International Joint Conference on Artificial Intelligence (IJCAI)* (2019), pp. 4739–4745.
27. H. Guo, S. Yuan, and X. Wu, "LogBERT: log anomaly detection via BERT," in *International Joint Conference on Neural Networks (IJCNN)* (2021).
28. V. H. Le and H. Zhang, "Log parsing: how far can ChatGPT go?" in *38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2023), pp. 1699–1704.
29. Z. A. Khan, T. Yu, Y. Xu, *et al.*, "Guidelines for assessing the accuracy of log message template identification techniques," in *44th International Conference on Software Engineering* (2020), pp. 1095–1106.
30. Z. Jiang, J. Liu, J. Huang, *et al.*, "A large-scale evaluation for log parsing techniques: how far are we?" in *33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (2024).
31. C. Almodovar, F. Sabrina, S. Karimi, *et al.*, "LogFit: log anomaly detection using fine-tuned language models," *IEEE Trans. Netw. Serv. Manage.* **21**, 1715–1723 (2024).
32. C. Jeong, "Generative AI service implementation using LLM application architecture: based on RAG model and LangChain framework," *J. Intell. Inf. Syst.* **29**, 129–164 (2023).
33. M. Shu, J. Wang, C. Zhu, *et al.*, "On the exploitability of instruction tuning," in *Advances in Neural Information Processing Systems* (2024).
34. S. Longpre, L. Hou, T. Vu, *et al.*, "The flan collection: designing data and methods for effective instruction tuning," in *International Conference on Machine Learning* (2023), pp. 22631–22648.
35. T. Detrmers, A. Pagnoni, A. Holtzman, *et al.*, "QLoRA: efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems* (2024).
36. T. T. Nguyen, C. Wilson, and J. Dalins, "Fine-tuning LLaMA 2 large language models for detecting online sexual predatory chats and abusive texts," *arXiv* (2023).
37. O. Topsakal and T. C. Akinci, "Creating large language model applications utilizing LangChain: a primer on developing LLM apps fast," in *International Conference on Applied Engineering and Natural Sciences* (2023), pp. 1050–1056.
38. A. Singh, A. Ehtesham, S. Mahmud, *et al.*, "Revolutionizing mental health care through LangChain: a journey with a large language model," in *IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)* (2024), pp. 73–78.
39. Z. M. Jiang, A. E. Hassan, P. Flora, *et al.*, "Abstracting execution logs to execution events for enterprise applications (short paper)," in *8th International Conference on Quality Software* (2008), pp. 181–186.
40. A. Parnami and M. Lee, "Learning from few examples: a summary of approaches to few-shot learning," *arXiv* (2022).
41. spaCy: <https://github.com/explosion/spaCy>.
42. W. Xu, L. Huang, A. Fox, *et al.*, "Detecting large-scale system problems by mining console logs," in *ACM SIGOPS 22nd Symposium on Operating Systems Principles* (2009), pp. 117–132.
43. Y. Li, Z. Li, K. Zhang, *et al.*, "ChatDoctor: a medical chat model fine-tuned on a large language model meta-AI (LLaMA) using medical domain knowledge," *Cureus* **15**, e40895 (2023).
44. H. Du, G. Liu, Y. Lin, *et al.*, "Mixture of experts for network optimization: a large language model-enabled approach," *arXiv* (2024).
45. Y. Huang, L. J. Wan, H. Ye, *et al.*, "New solutions on LLM acceleration, optimization, and application," *arXiv* (2024).
46. Y. Wang, H. Ivison, P. Dasigi, *et al.*, "How far can camels go? Exploring the state of instruction tuning on open resources," *arXiv* (2023).
47. B. Hartmann, P. Tamla, F. Freund, *et al.*, "Fine-tune it like I'm five: supporting medical domain experts in training NER models using cloud, LLM, and auto fine-tuning," in *31st Irish Conference on Artificial Intelligence and Cognitive Science (AICS)* (2023).
48. Z. Ji, T. Yu, Y. Xu, *et al.*, "Towards mitigating LLM hallucination via self reflection," in *Findings of the Association for Computational Linguistics: EMNLP* (2023), pp. 1827–1843.
49. X. Jiang, M. Zhang, Y. Song, *et al.*, "OptiComm-GPT: a GPT-based versatile research assistant for optical fiber communication systems," *Opt. Express* **32**, 20776–20796 (2024).
50. D. Wang, Y. Wang, X. Jiang, *et al.*, "When large language models meet optical networks: paving the way for automation," *arXiv* (2024).